

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

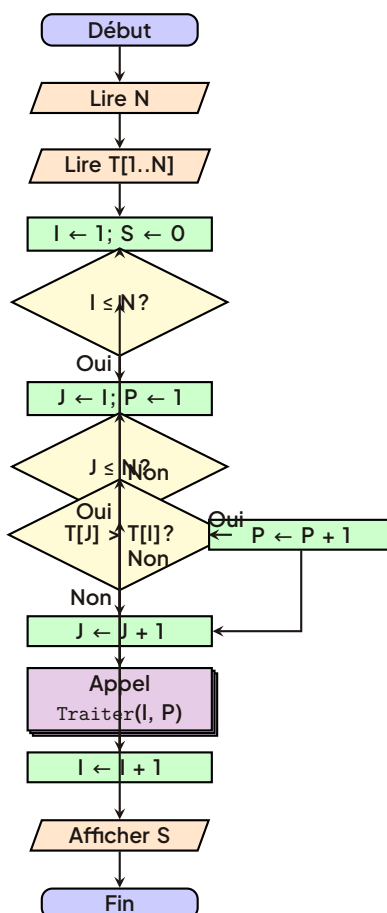
Construction d'algorigrammes et exécution des algorithmes à la main

Module 1 : Algorithmique

Cette fiche propose des exercices de synthèse de niveau Baccalauréat, mobilisant le tracé, l'analyse de complexité en instructions, la récursivité et la manipulation de structures de données.

EXERCICE 1 Analyse d'un algorithme imbriqué et comptage d'instructions

On considère l'algorithme ci-dessous, traitant un tableau T d'entiers de taille $N \geq 1$. Les symboles utilisés respectent la norme (ovale : début/fin, parallélogramme : entrée/sortie, rectangle : affectation, losange : test, rectangle à double barre : sous-algorithme).



Le sous-algorithme $\text{Traiter}(x, y)$ effectue : $S \leftarrow S + (x * y)$.

- Traduire fidèlement cet algorithme en algorithme textuel (pseudo-code normalisé).

2. Déterminer le nombre exact d'instructions élémentaires (affectations, tests, appels, E/S) exécutées en fonction de N pour le pire des cas. Justifier.
3. Quel est le rôle global de l'algorithme principal vis-à-vis du tableau T ? Expliquer la logique de la boucle interne sur J .
4. Proposer une modification de l'algorithme (ajout/modification de symboles) pour que la variable S contienne à la fin la somme des maximums de tous les sous-tableaux $T[I..N]$. Justifier sans réécrire tout l'algorithme.

EXERCICE 2 Récursivité, pile d'exécution et terminaison

Soit la fonction récursive suivante définie sur les entiers naturels :

```
Fonction F(n : Entier) : Entier
Début
    Si n = 0 Alors Renvoyer 1
    Sinon Si n = 1 Alors Renvoyer 2
    Sinon Renvoyer 3 * F(n-1) - 2 * F(n-2)
    Fin Si
Fin
```

1. Dessiner l'arbre d'appels récursifs pour le calcul de $F(4)$. Indiquer la valeur de retour sur chaque nœud.
2. Simuler l'état de la pile d'exécution (contenu des cadres d'activation : paramètre n , adresse de retour, variables locales) au moment où l'appel $F(0)$ est atteint pour la première fois lors du calcul de $F(3)$.
3. Démontrer par récurrence forte que $\forall n \in \mathbb{N}, F(n) = 2^n + 1$.
4. Écrire une version itérative de cette fonction utilisant une boucle et un nombre constant de variables. Comparer le nombre d'opérations arithmétiques entre les deux versions pour calculer $F(n)$.

EXERCICE 3 Exécution pas à pas avec structure de données : Tri par insertion sur liste chaînée

On représente une liste chaînée d'entiers par un tableau de nœuds $\text{Noeud}[1..Max]$ où chaque nœud a deux champs : Val (entier) et Suiv (indice du suivant, 0 si fin). La liste est pointée par Tete (indice du premier nœud, 0 si vide). On dispose de la procédure $\text{InsérerTrié}(\text{Ref Tete}, \text{Val } x)$ ci-dessous :

```
Procédure InsérerTrié(Ref Tete, Val x)
```

```

Var Cour, Prec, Nouv : Entier
Début
    Nouv <- Allouer() // Renvoie indice libre ou -1 si plein
    Si Nouv = -1 Alors Fin // Échec allocation
    Noeud[Nouv].Val <- x
    Si Tete = 0 OU x < Noeud[Tete].Val Alors
        Noeud[Nouv].Suiv <- Tete
        Tete <- Nouv
    Sinon
        Prec <- Tete
        Cour <- Noeud[Tete].Suiv
        TantQue Cour <> 0 ET Noeud[Cour].Val < x Faire
            Prec <- Cour
            Cour <- Noeud[Cour].Suiv
        Fin TantQue
        Noeud[Nouv].Suiv <- Cour
        Noeud[Prec].Suiv <- Nouv
    Fin Si
Fin

```

On part d'une liste vide ($Tete = 0$). On exécute séquentiellement : $InsererTrié(Tete, 20)$, $InsererTrié(Tete, 10)$, $InsererTrié(Tete, 30)$, $InsererTrié(Tete, 5)$.

1. Pour chaque appel, tracer l'état complet de la mémoire (tableau `Noeud` indices 1 à 4, variable `Tete`) après l'exécution de l'appel. Présenter sous forme de tableau.
2. Après ces 4 insertions, donner la séquence des valeurs obtenues en parcourant la liste depuis `Tete`.
3. Identifier l'invariant de la boucle `TantQue` dans le bloc `Sinon`. Prouver qu'il est initialisé, conservé et qu'il permet d'affirmer la correction de l'insertion (ordre croissant maintenu).
4. Analyser le cas où `Allouer()` renvoie -1 (mémoire pleine). L'algorithme laisse-t-il la liste dans un état cohérent ? Justifier.

EXERCICE 4 Construction d'algorithme : Partition de tableau (Algorithme de Hoare)

On souhaite construire l'algorithme de la procédure `Partitionner(T, g, d, Ref p)` qui réorganise le tableau $T[g..d]$ (indices entiers, $g < d$) autour d'un pivot $T[g]$ (stratégie de Hoare), de sorte qu'à la fin : $T[g..p-1] \leq \text{pivot} < T[p+1..d]$ et $T[p] = \text{pivot}$. La procédure renvoie l'indice final p du pivot.

1. Déclarer les variables locales nécessaires (indices, pivot, temporaire).
2. Construire l'algorithme complet et structuré (un seul entrée, une seule sortie) cor-

respondant à l'algorithme suivant :

```
pivot <- T[g] ; i <- g ; j <- d + 1
Répéter
  Répéter i <- i + 1 Jusqu'à (i > d) OU (T[i] >= pivot)
  Répéter j <- j - 1 Jusqu'à (T[j] <= pivot)
  Si i < j Alors Échanger(T[i], T[j]) Fin Si
Jusqu'à i >= j
Échanger(T[g], T[j]) ; p <- j
```

3. Sur l'algorithme, identifier explicitement (entourer/annoter) les zones correspondant aux : initialisation, boucles internes, boucle principale, finalisation.
4. Tracer l'exécution de cet algorithme pour $T = [5, 3, 8, 4, 2, 7, 1, 6]$, $g=1$, $d=8$. Donner l'état final du tableau et la valeur de p .